**DEPARTMENT OF COMPUTER & INFORMATION SYSTEMS ENGINEERING**
**BACHELORS IN COMPUTER SYSTEMS ENGINEERING**
**Course Code: CS-116**
**Course Title: Object Oriented Programming**
**Complex Engineering Problem**
**FE Batch 2025, Spring Semester 2026**
**Grading Rubric**
**TERM PROJECT**

| Student No. | Name | Roll No. |
|---|---|---|
| S1 | **Muhammad Mustafa** | CS-25072 |
| S2 | **Syeda Sameen Fatima** | CS-25066 |
| S3 | **Waheed -Ul Haq** | CS-25070 |
| S4 | | |

| **CRITERIA AND SCALES** | | | | **Marks Obtained** | | | |
|---|---|---|---|---|---|---|---|
| | | | | **S1** | **S2** | **S3** | **S4** |
| Criterion 1: Does the application meet the desired specifications and produce the desired outputs? (CPA-1, CPA-3) | | | | | | | |
| 1 | 2 | 3 | 4 | | | | |
| The application does not meet the desired specifications and is producing incorrect outputs. | The application partially meets the desired specifications and is producing incorrect or partially correct outputs. | The application meets the desired specifications but is producing incorrect or partially correct outputs. | The application meets all the desired specifications and is producing correct outputs. | | | | |
| Criterion 2: How well is the code organization? | | | | | | | |
| 1 | 2 | 3 | 4 | | | | |
| The code is poorly organized and very difficult to read. | The code is readable only to someone who knows what it is supposed to be doing. | Some part of the code is well organized, while some part is difficult to follow. | The code is well organized and very easy to follow. | | | | |
| Criterion 3: How friendly is the application interface? (CPA-1, CPA-3) | | | | | | | |
| 1 | 2 | 3 | 4 | | | | |
| The application interface is difficult to understand and use. | The application interface is easy to understand and but not that comfortable to use. | The application interface is very easy to understand and use. | The application interface is very interesting/ innovative and easy to understand and use. | | | | |
| Criterion 4: How does the student performed individually and as a team member? (CPA-2, CPA-3) | | | | | | | |
| 1 | 2 | 3 | 4 | | | | |
| The student did not work on the assigned task. | The student worked on the assigned task, and accomplished goals partially. | The student worked on the assigned task, and accomplished goals satisfactorily. | The student worked on the assigned task, and accomplished goals beyond expectations. | | | | |
| Criterion 5: Does the report adhere to the given format and requirements? | | | | | | | |
| 1 | 2 | 3 | 4 | | | | |
| The report does not contain the required information and is formatted poorly. | The report contains the required information only partially but is formatted well. | The report contains all the required information but is formatted poorly. | The report contains all the required information and completely adheres to the given format. | | | | |
| | | | Total Marks: | | | | |

# Table of Contents

# 1. Problem Description

The **Library and Resource Management System** is designed to automate the manual processes of a library, specifically tracking resource availability, member borrowing limits, and fine calculations. The system addresses the need for a centralized platform where administrators can manage an inventory of books (add, update, or remove) and students can seamlessly search for, issue, and return resources.

A unique challenge addressed in this project is the integration of the **Open Library API** using curl, allowing users to search for books globally and request the administrator to add specific titles via their ISBN or title. The system also enforces strict business logic, such as a 3-book borrowing limit and automated daily fine generation for overdue items, ensuring organized and efficient library operations.

---

| System Layer | Functionality & Modules | Technical Implementation |
|---|---|---|
| **I. Presentation Layer** | User & Admin Dashboards, Menu interfaces (`menu`, `mmenu`) | Handles CLI interactions and provides a user-friendly console interface. |
| **II. Logic Layer** | Borrowing rules, Fine calculation (`calculatefine`), Exception Handling | Enforces tiered limits (2, 5, or 7 books) and credential validation. |
| **III. Data & Integration Layer** | File Handling (`books.txt`, `users.txt`, `records.txt`, `reservations.txt`, `requests.txt`), Open Library API Integration. | Manages persistent storage and fetches external data using the `curl` command. |

## 2. Distinguishing Features & OOP Concepts

This project demonstrates the application of complex engineering attributes by integrating real-time data with fundamental Object-Oriented principles. The following features are implemented:

1. **Hierarchical Inheritance**: The system utilizes hierarchical inheritance where both the **USER** and **ADMIN** classes are derived from an abstract base class called **person**. This structure allows for a clean implementation of polymorphism through the pure virtual `display ()` function, which is overridden in both derived classes.

2. **Encapsulation**: Sensitive data members, such as user passwords, account balances, and resource availability, are kept **private** or **protected** . These are accessed strictly through public getter and setter functions to ensure data integrity and security .

3. **Robust Input Validation**: To prevent system crashes and infinite loops, the system replaces standard `cin` with specialized helper functions:

    i. **checkdigit ()**: Validates that the input is a numeric integer, preventing errors during menu selection.

    ii. **checkDouble ()**: Ensures decimal inputs are valid for financial transactions like adding balance or paying fines.

    iii. **checkusername () & checkpass ()**: Enforces strict security protocols, including an 8-character minimum and special character requirements for credentials.

4. **Tiered Membership Logic**: The system implements a dynamic logic layer where borrowing limits and due dates are determined by the user's membership tier:

    i. **Basic Plan**: 2 books and a 10-day due date.

    ii. **Silver Plan**: 5 books and a 30-day due date .

    iii. **Gold Plan**: 7 books and a 60-day due date.

5. **Custom Exception Handling**: A robust error management system is built using a hierarchy of custom exception classes derived from `std::exception` . Specific exceptions such as

`userexception`, `booknotfound`, and `fineexception` allow the system to provide user-friendly feedback without terminating the program.

6. **Association (Aggregation/Composition)**: The **LIBRARY** class acts as a central manager, utilizing STL vectors to store and synchronize objects of **RESOURCE**, **USER**, **BORROWRECORD**, and **RESERVATION**.

7. **Data Persistence**: Persistent storage is achieved through advanced file I/O operations. The system serializes and deserializes object data across five distinct files: `books.txt`, `users.txt`, `records.txt`, `reservations.txt`, and `requests.txt`.

8. **External API Integration (Unique Feature)**: A distinguishing technical achievement is the integration of the **Open Library API**. Using Windows `curl` commands, the system fetches real-time JSON metadata for books. The system performs manual string parsing using `.find()` and `.substr()` to extract title and author information without external JSON libraries .

## 3. Flow of Project

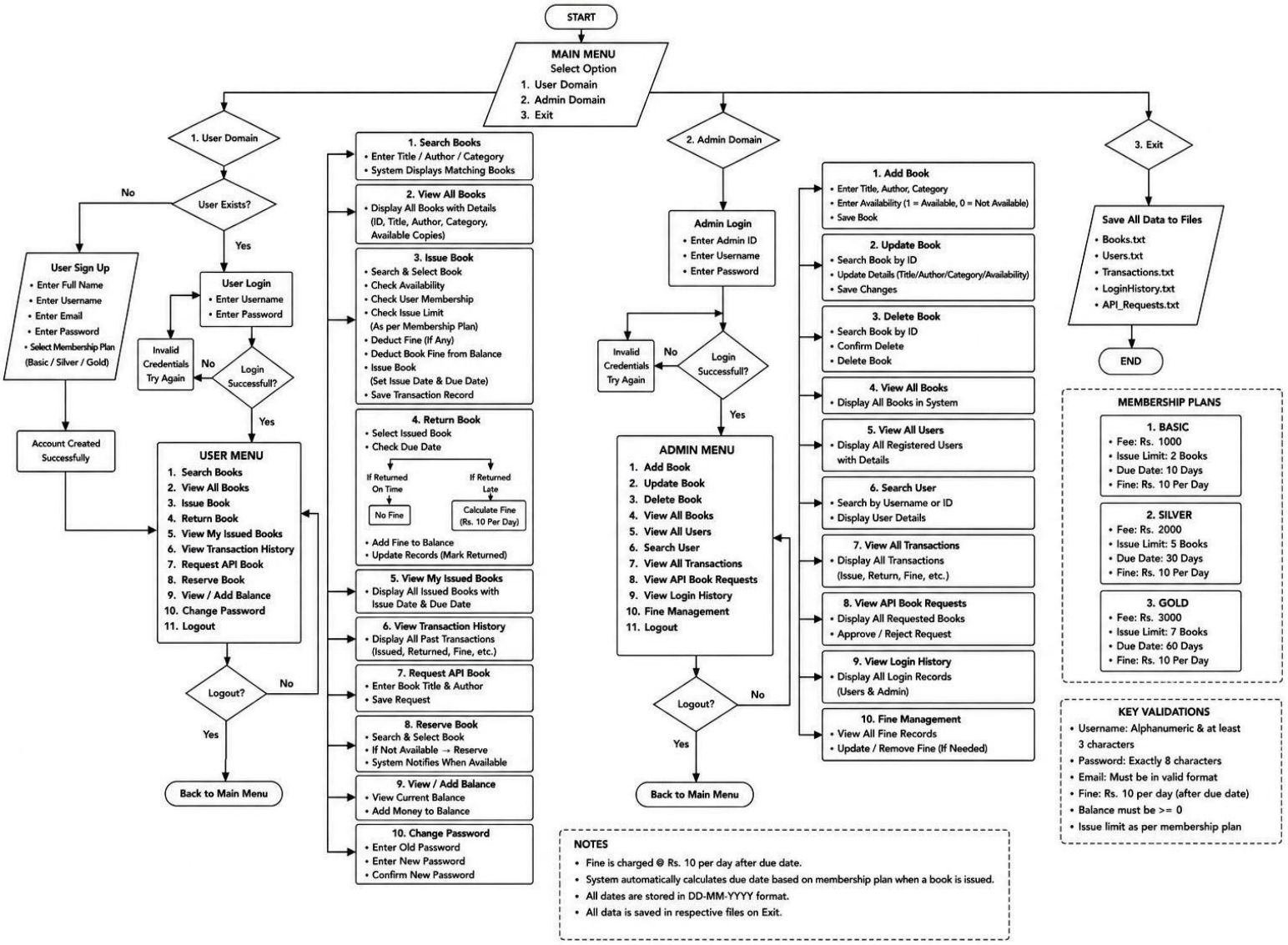
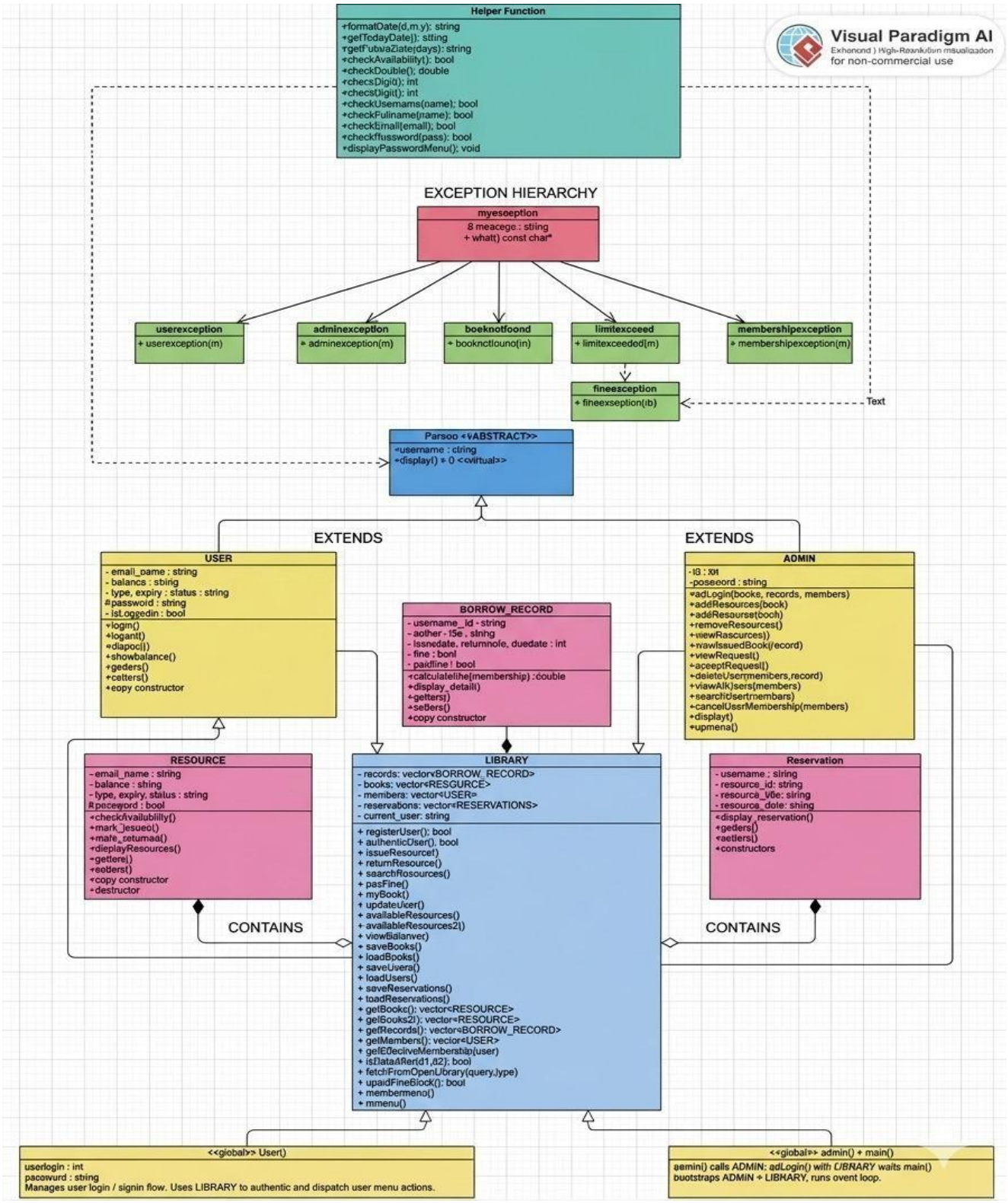



*Figure 1: Overall Workflow of the Library Management System*

### Description

The flowchart above represents the complete workflow of the Library Management System. The system is divided into two main domains: User Domain and Admin Domain. Users can register, log in, search books, issue or return resources, reserve unavailable books, manage balances, and request books through ***Open Library API integration.*** The Admin Domain manages books, users, transactions, API requests, login history, and fine management. The system also includes validation checks, ***membership-based borrowing policies***, automatic fine calculation, and file handling for permanent data storage.

# UML


Helper Function
+formatDate(d,m,y): string
+getTodayDate(): string
+getFutureDate(days): string
+checkAvailability(): bool
+checkDouble(): double
+checkDigit(): int
+checkDigit(): int
+checkUsername(name): bool
+checkFullname(name): bool
+checkEmail(email): bool
+checkPassword(pass): bool
+displayPasswordMenu(): void
Visual Paradigm AI
for non-commercial use
EXCEPTION HIERARCHY
myexception
& message : string
+ what() const char*
userexception
+ userexception(m)
adminexception
+ adminexception(m)
booknotfound
+ booknotfound(in)
limitexceed
+ limitexceeded(m)
membershipexception
+ membershipexception(m)
fineexception
+ fineexception(ib)
Text
Person <<ABSTRACT>>
#username : string
#display() = 0 <<virtual>>
EXTENDS
EXTENDS
USER
- email_name : string
- balance : string
- type, expiry : status : string
#password : string
- isLoggedin : bool
+login()
+logout()
+display()
+showbalance()
+getters()
+setters()
+copy constructor
BORROW_RECORD
- username_id - string
- author - title , string
- issuedate, returnhole, duedate : int
- fine : bool
- paidfine ! bool
+calculatefine(membership) :double
+display_detail()
+getters()
+setters()
+copy constructor
ADMIN
- id : int
-password : string
+adLogin(books, records, members)
+addResources(book)
+addResources(book)
+removeResources()
+viewRascurces()
+viewIssuedBook(record)
+viewRequest()
+aceeptRequest()
+deleteUser(members,record)
+viewAllUsers(members)
+searchUser(members)
+cancelUserMembership(members)
+display()
+upmenu()
RESOURCE
- email_name : string
- balance : string
- type, expiry, status : string
#password : bool
+checkAvailability()
+mark_issued()
+mark_returned()
+displayResources()
+getters()
+setters()
+copy constructor
+destructor
CONTAINS
LIBRARY
- records: vector<BORROW_RECORD>
- books: vector<RESOURCE>
- members: vector<USER>
- reservations: vector<RESERVATIONS>
- current_user: string
+ registerUser(): bool
+ authenticUser(): bool
+ issueResource()
+ returnResource()
+ searchResources()
+ pasFine()
+ myBook()
+ updateUser()
+ availableResources()
+ availableResources2()
+ viewBalance()
+ saveBooks()
+ loadBooks()
+ saveUsers()
+ loadUsers()
+ saveReservations()
+ loadReservations()
+ getBooks(): vector<RESOURCE>
+ getBooks2(): vector<RESOURCE>
+ getRecords(): vector<BORROW_RECORD>
+ getMembers(): vector<USER>
+ getEffectiveMembership(user)
+ isDateAfter(d1,d2): bool
+ fetchFromOpenLibrary(query,type)
+ unpaidFineBlock(): bool
+ membermenu()
+ mmenu()
CONTAINS
Reservation
- username : string
- resource_id: string
- resource_title: string
- resource_date: string
+display_reservation()
+getters()
+setters()
+constructors
<<global>> User()
userlogin : int
password : string
Manages user login / signin flow. Uses LIBRARY to authentic and dispatch user menu actions.
<<global>> admin() + main()
admin() calls ADMIN::adLogin() with LIBRARY waits main()
bootstraps ADMIN + LIBRARY, runs event loop.

# 4. Most Challenging Part

## 4.1 Manual Data Extraction from External API

The most significant technical hurdle was implementing the **Open Library API integration** without an external JSON library. Since the curl command returns a raw string of metadata, extracting specific fields required extreme precision.

- **The Challenge:** The raw JSON string contained a massive amount of unformatted text. Using standard cin or simple parsing was impossible because the data structure was dynamic.
- **The Solution:** I utilized the. find () and. substr () methods to identify unique "bibkeys" and markers. By calculating precise offsets, I was able to extract the "Title," "Author," and "ISBN" directly from the buffer, ensuring the system remains lightweight.

## 4.2 Mathematical Synchronization of Date Strings

The fine calculation is synchronized by parsing date strings into struct tm objects. The logic follows a base rate of **10 PKR per day** with the following membership modifiers:

- **Gold Members**: 50% discount on total fines.
- **Silver Members**: 20% discount on total fines.
- **Basic Members**: Fine is doubled if the delay exceeds 10 days.

## 5. Any New Thing Learnt in C++

1. **Robust Input Validation & Buffer Management:**
   Implemented reliable input handling in C++ using cin.fail(), cin.clear(), and cin.ignore() to prevent crashes caused by invalid user input. Developed structured buffer control techniques to ensure smooth and user-friendly program execution.
2. **Data Persistence & File Stream Serialization:**
   Designed and implemented persistent storage mechanisms by serializing object states (e.g., user balances and system resources) into external .txt files. Enabled data retention across sessions, reflecting real-world enterprise application practices.
3. **Dynamic Memory Management using STL Vectors:**
   Replaced static arrays with STL vectors integrated within object-oriented structures. Leveraged dynamic memory allocation to support scalable data handling and efficient runtime database management.
4. **API Integration & Networking Fundamentals:**
   Integrated external APIs into a console-based C++ application using curl and system-level commands. Retrieved and processed real-time data from the Open Library API, demonstrating foundational networking and software integration skills.
5. **Multi-Source Data Synchronization:**
   Developed logic to synchronize local file-based data with real-time API responses. Ensured consistency between static and dynamic data streams, aligning with modern application design patterns.

## 6. Individual Contributions

This section outlines the distribution of work among team members for the development of the Object-Oriented Programming (OOP) based Library Management System. The project follows a collaborative development approach, where each member contributed to different phases including design, implementation, testing, and documentation. This division of work ensured efficient workflow, better code organization, and a successful completion of the system aligned with real-world software development practices.

| Member | Roll No. | Responsibilities / Contributions |
|---|---|---|
| Muhammad Mustafa | CS-25072 | Played a key role in core system development, including implementation of major functionalities and program logic. Collaborated in coding and system integration tasks. |
| Syeda Sameen Fatima | CS-25066 | Contributed to both development and documentation. Assisted in core coding tasks and took responsibility for report writing, formatting, and overall documentation structure. |
| Waheed-ul-haq | CS-25070 | Responsible for system testing and debugging. Executed test cases, identified logical and runtime errors, and ensured overall system reliability and correctness. |

All members collaborated during the final integration phase to ensure proper system functionality and successful completion of project requirements.

# 7. Future Enhancements / Future Scope

### 1. Graphical User Interface (GUI) Development

The current console-based system can be improved by adding a graphical interface using frameworks such as Qt or SFML. This would make the system more interactive, visually appealing, and easier for users to operate.

### 2. Database Integration

Instead of file handling, the system can be upgraded to a database system like MySQL or SQLite. This would improve data management, increase security, and allow efficient handling of large amounts of library data.

### 3. Advanced User Roles and Security System

A more advanced role-based access system can be introduced where different users (students, librarians, and administrators) have different permissions. This would enhance system security and control.

### 4. Smart Recommendation System

A future improvement can include a recommendation feature that suggests books to users based on their search history or previously issued books, making the system more intelligent and user-friendly.

### 5. Cloud Storage and Online Synchronization

The system can be extended to support cloud-based storage so that all records are synchronized online. This would allow access from multiple devices and ensure data availability at all times.

### 6. Automated Notifications System

An automated system can be implemented to notify users about due dates, overdue fines, or book availability through email or SMS, improving user engagement and system efficiency.

### 7. Barcode / QR Code Integration

Books can be assigned QR codes or barcodes to simplify issuing and returning processes, reducing manual errors and making the system more practical for real-world library environments.

**8. System Architecture and Class Relationships**

The Library and Resource Management System is built on a modular architecture where distinct classes handle specific business logic, data persistence, and user interactions.

# 8. Test Cases

## 9.1 Open Library API Integration

To verify that the system successfully searches books online using the Open Library API, sends a request to the admin, and allows the admin to approve and add the requested book into the library system.

```
Enter your choice : 4
SEARCH ONLINE BOOK (OPEN LIBRARY API)
1- Search by TITLE
2- Search by AUTHOR
3- Search by ISBN
Select the Above Option : 1
Enter Title Name : python
Connecting to Open Library...

===== CLEAN SEARCH RESULT =====
1. Learning Python| Mark Lutz| 13
-----------------------
2. Python| Allen B. Downey| 11
-----------------------
3. Think Python| start| {"numFound":4765
-----------------------
4. Learning Python| Mark Lutz| 13
-----------------------
5. Python| Allen B. Downey| 11
-----------------------
Select Book Number : 1

Do you want to REQUEST this book from Admin? (y/n): y
Request sent to Admin!
```

*Figure 2 : API-Based Book Search and Request Submission*

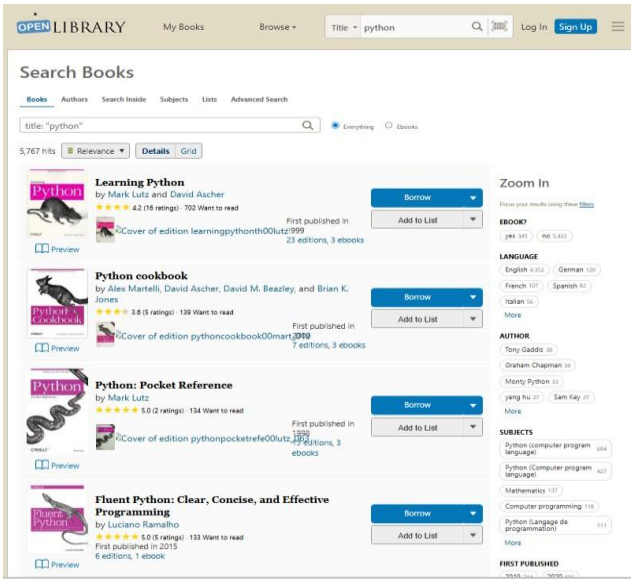



*Figure 3: Open Library API Interface*

```
----------------Library Management System--------------
1- USER DOMAIN
2- ADMIN DOMAIN
3- EXIT
Please Select from the above option : 2
Enter The Admin ID : 786
Enter The Username : admin
Enter The Admin Password : Admin123
Login Successfull
1- Add Resource
2- Remove Resource
3- Update Resource
4- View All Resource
5- View All Issued Resource
6- View All Student Request
7- Accept Student Request
8- Delete User
9- View All Users
10- Search User
11- Cancel Memebership
12- Exit
Please Select from the above option : 6

===== STUDENT BOOK REQUESTS =====
1. sameen requested: Learning Python, Mark Lutz, 13
2. mustafa requested: Beginning Web programming with HTML, XHTML, and CSS, Ben Frain, 5
3. waheed requested: Effective Java, start, {"numFound":8261
==================================
```

*Figure 4: admin can see all requests*

```
1- Add Resource
2- Remove Resource
3- Update Resource
4- View All Resource
5- View All Issued Resource
6- View All Student Request
7- Accept Student Request
8- Delete User
9- View All Users
10- Search User
11- Cancel Memebership
12- Exit
Please Select from the above option : 7
1. sameen requested: Learning Python, Mark Lutz, 13
2. mustafa requested: Beginning Web programming with HTML, XHTML, and CSS, Ben Frain, 5
3. waheed requested: Effective Java, start, {"numFound":8261
Which Request Do you want to select : 1
Processing : sameen requested: Learning Python, Mark Lutz, 13
Enter Resouce ID : CS-123
Enter the Title Name : learning python
Enter the Author Name : Mark Lutz
Enter the Category : computer
Successfully the Book is being Updated!
```

*Figure 5: Admin can accept any request*

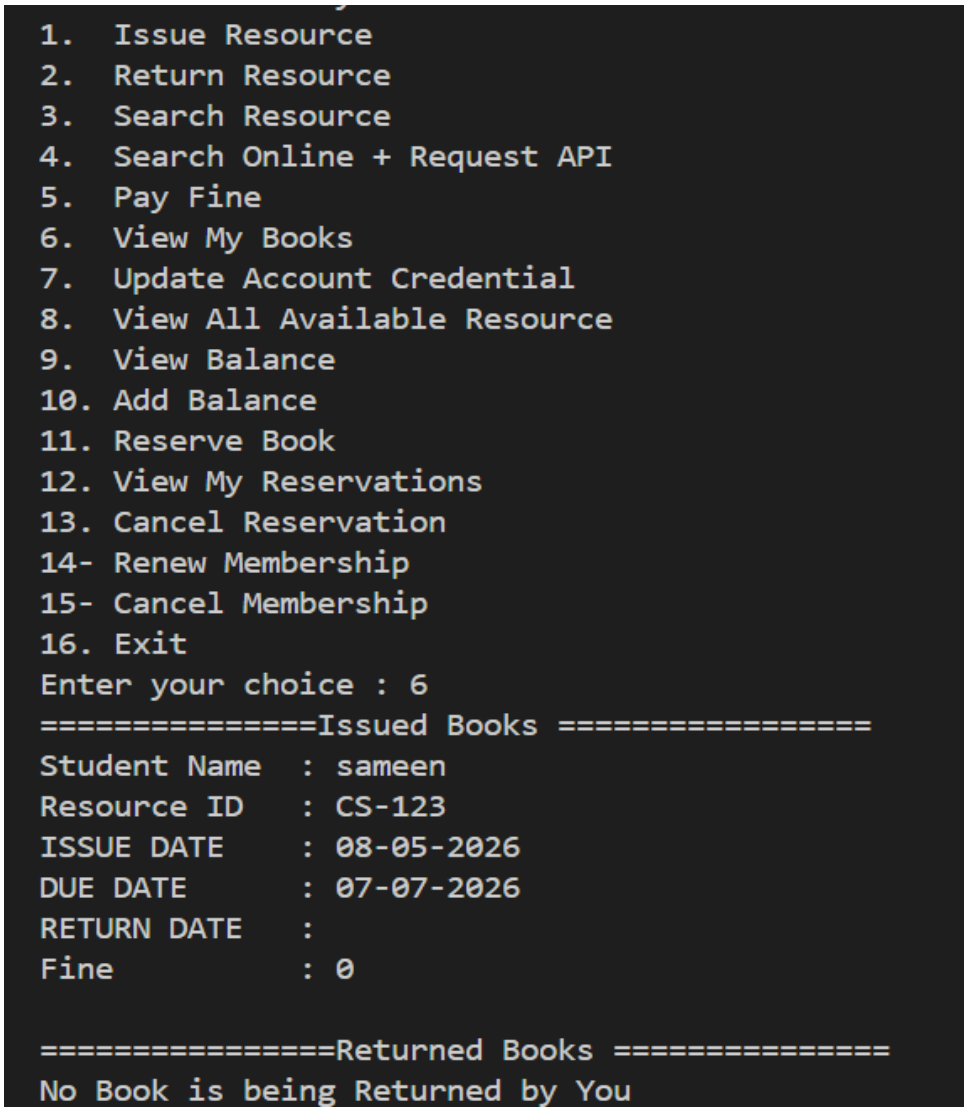



*Figure 6: book issued successfully*

## Description

The system successfully integrated the Open Library API to search books online. After the user requested a book, the request was stored and displayed to the admin through the request management system. The admin approved the request and added the resource into the library database, after which the user was able to issue the book successfully. This feature demonstrates API integration, file handling, admin-user interaction, and resource management working together efficiently.

### 9.2 Reservation & Auto-Issue System

To verify that reserved books are automatically issued when they become available.

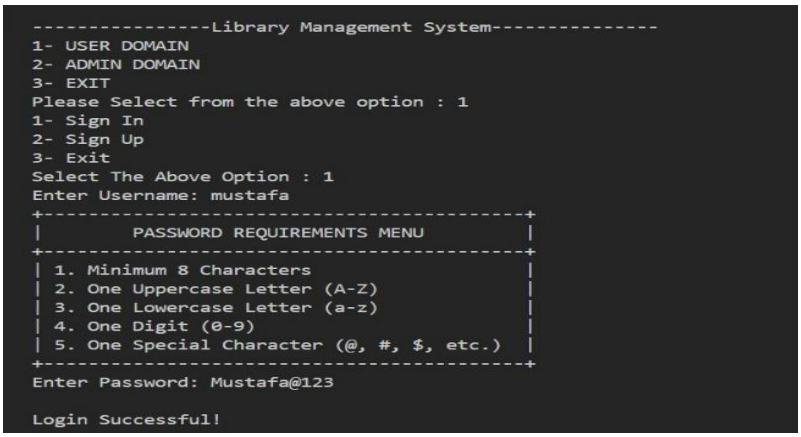



*Figure 7: Mustafa(user) logged in*

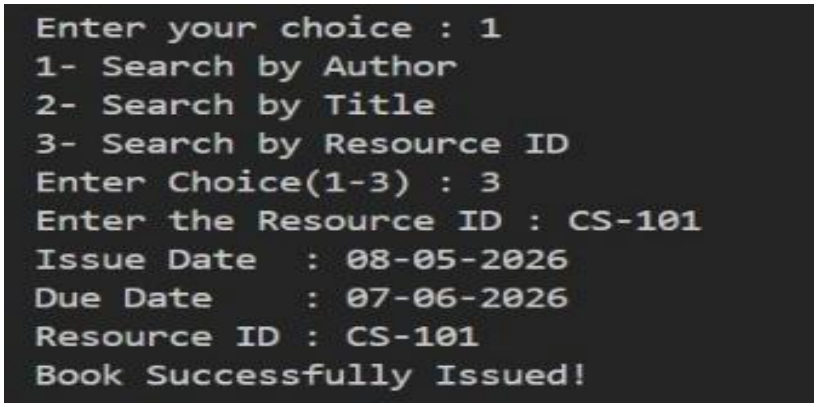



*Figure 8: Mustafa issued book CS-101*

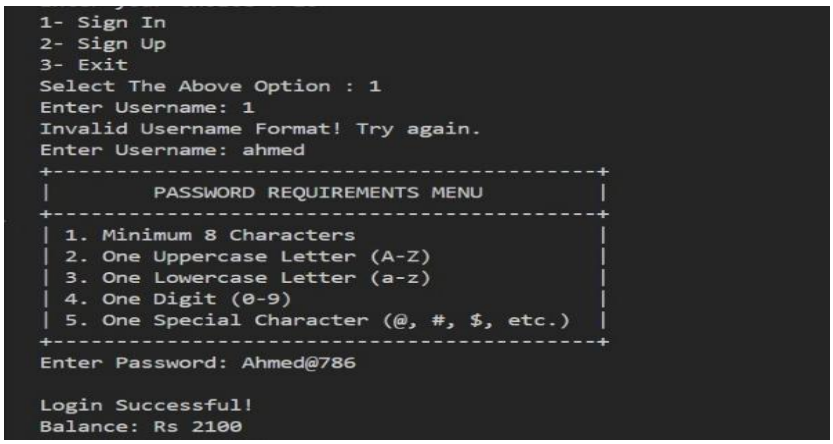



*Figure 9: Ahmed(user)logged in*

```
11. Reserve Book
12. View My Reservations
13. Cancel Reservation
14- Renew Membership
15- Cancel Membership
16. Exit
Enter your choice : 11
Enter the Resource ID to Reserve : CS-101

[RESERVED] Book is currently unavailable.
Will be auto-issued once returned by current holder.
```

*Figure 10: Ahmed reserved CS-101*

```
Enter your choice : 2
Enter the Resource ID : CS-101
Return Date : 08-05-2026

========================================
[AUTO-ISSUED] 'CS-101' issued to 'ahmed'
Issue Date : 08-05-2026  |  Due Date : 07-06-2026
========================================

Student Name  : mustafa
Resource ID   : CS-101
ISSUE DATE    : 08-05-2026
DUE DATE      : 07-06-2026
RETURN DATE   : 08-05-2026
Fine          : 0

Resource Returned Successfully
```

*Figure 11: Mustafa returned CS-101 and CS-101 auto issued to Ahmed*

## Description

The system successfully handled book reservation and automatically issued the resource to the reserved user after the previous user returned it. This feature demonstrates automated reservation handling and real-time resource availability management.

## 9.3 Membership-Based Borrowing Rules

### Objective

To verify that different membership plans enforce different issue limits and due dates.

### Input

```
Membership Type = Gold
Books Already Issued = 7
Attempt to issue another book
```

```
----------------Library Management System---------------
1- USER DOMAIN
2- ADMIN DOMAIN
3- EXIT
Please Select from the above option : 1
1- Sign In
2- Sign Up
3- Exit
Select The Above Option : 2
Enter Username: sameen
Enter the Full Name : syeda sameeen
+-----------------------------------------+
|        PASSWORD REQUIREMENTS MENU       |
+-----------------------------------------+
| 1. Minimum 8 Characters                 |
| 2. One Uppercase Letter (A-Z)           |
| 3. One Lowercase Letter (a-z)           |
| 4. One Digit (0-9)                      |
| 5. One Special Character (@, #, $, etc.)  |
+-----------------------------------------+
Enter Password: Sameen_123
Enter Email: sameeen1374@gmail.com
====================MEMBERSHIP DETAILS=======================
1- BASIC PLAN      RS : 1000 | 2 Book Issue | 10 Days Due Date | 2 Reserved Book
2- SILVER PLAN     RS : 2000 | 5 Book Issue | 30 Days Due Date | 2 Reserved Book | 20% off on fine
3- GOLD PLAN       RS : 3000 | 7 Book Issue | 60 Days Due Date | 2 Reserved Book | 50% off on fine
Please Select The Above Option : 3
Enter the Initial Balance (min Rs 3000): 3000

[SUCCESS] User registered successfully!
Membership  : gold
Valid Until : 08-05-2027
```

*Figure 12: user created account with gold membership*

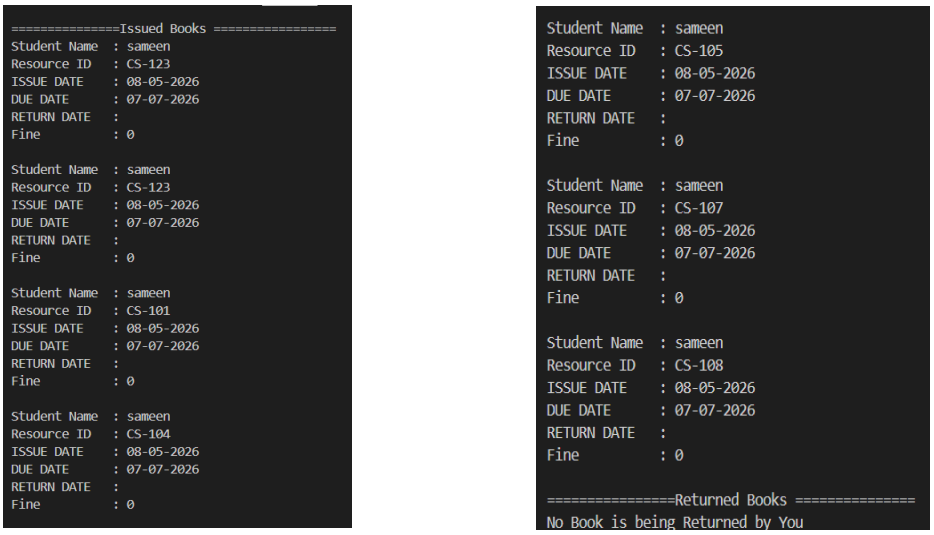



*Figure 13: user issued 7 books*

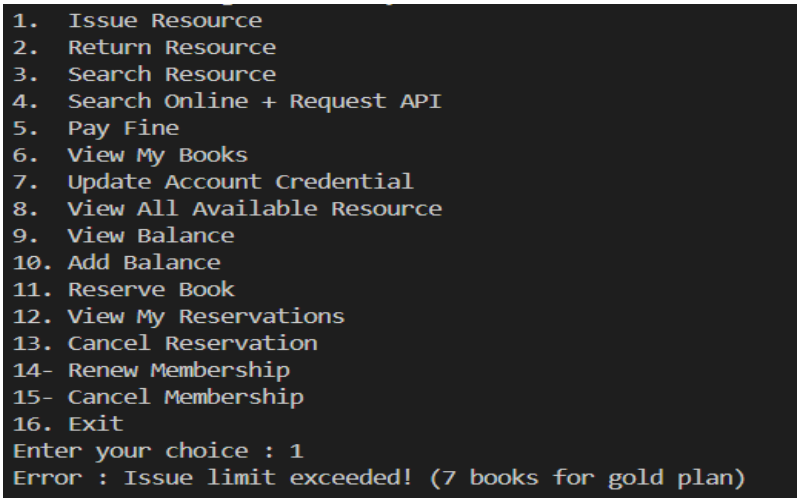



*Figure 14: failed attempt to issue another book*

## Description

The system correctly enforced borrowing limits according to the selected membership plan.

## 9.4 Custom Exception Handling

### Objective

To verify that custom exceptions handle invalid operations properly.

### User Registration Validation & Exception Handling

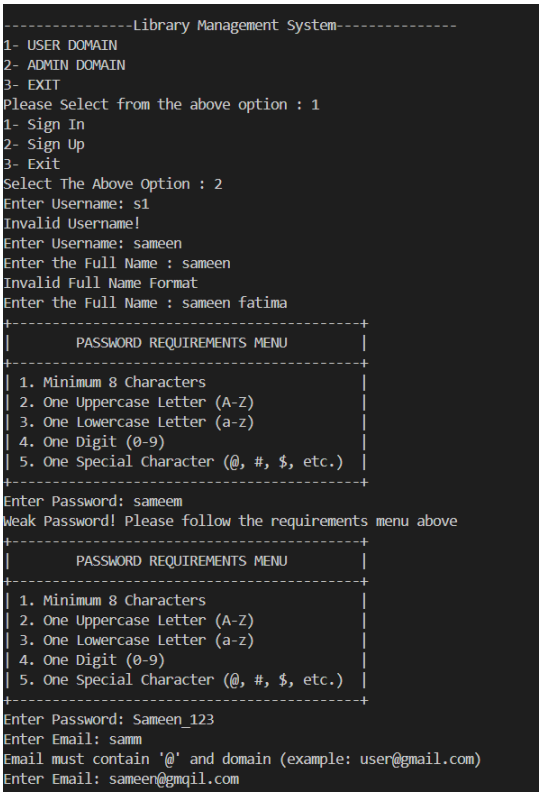



*Figure 15: registration validation*

### Invalid Admin Login Exception

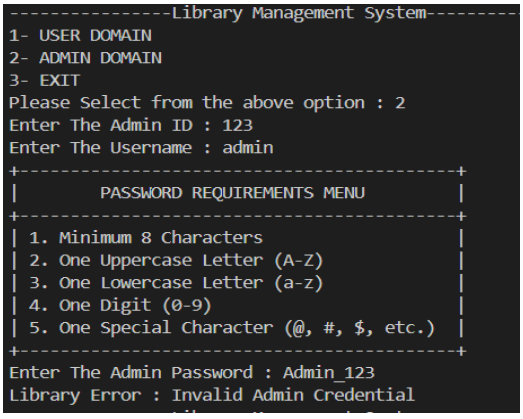



*Figure 16: Admin login exception*

**Book Not Found Exception**

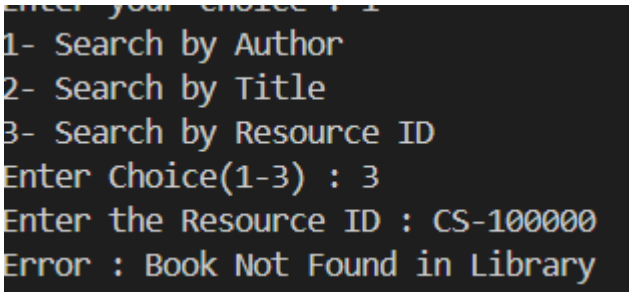



*Figure 17: book not fount exception*

## Description

The system performs strong input validation and exception handling to ensure data accuracy, security, and system reliability. Validation checks were implemented for username, full name, password strength, and email format during user registration.

If the user enters invalid information, the system immediately displays an appropriate error message and requests valid input again. Password validation ensures that the password contains:

- Minimum 8 characters
- Uppercase and lowercase letters
- Numeric digits
- Special characters

Similarly, email validation checks for proper email formatting including the presence of `@` and valid domain names.

In addition to registration validation, the system also handles different runtime exceptions such as:

- Invalid admin credentials during login
- Book not found in the library database
- Invalid user operations and access restrictions

These exception handling mechanisms improve the robustness, reliability, and security of the Library Management System by preventing invalid operations, unauthorized access, and incorrect data entries.

# *References*

**Open Library API Documentation**
https://openlibrary.org/developers/api
Used to understand the structure of the Open Library API and implement real-time book search functionality using external data integration in the system.

**C++ Standard Library Documentation (cppreference)**
https://en.cppreference.com/w/
Used for understanding and implementing core C++ features such as vectors, file handling, strings, and time manipulation used throughout the project.

**GeeksforGeeks – Object-Oriented Programming in C++**
https://www.geeksforgeeks.org/object-oriented-programming-in-cpp/
Used to strengthen understanding of OOP concepts such as classes, encapsulation, inheritance, and constructors used in the system design.

**Microsoft C++ Documentation**
https://learn.microsoft.com/en-us/cpp/cpp/
Used as a reference for proper usage of input/output streams, exception handling, and file processing techniques in C++.

**Stack Overflow Community**
https://stackoverflow.com/
Used for understanding error handling, debugging logic issues, and improving implementation of complex programming concepts during development.

**cURL Project Documentation** https://curl.se/docs/ Used as a reference for implementing command-line data transfer to fetch JSON metadata from external web servers within the C++ environment.

**UML Class Diagram Standards (Visual Paradigm)** https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-uml/ Used to ensure the class diagram correctly represents inheritance, association, and aggregation relationships according to engineering standards.

**Course Lectures & Resource Material Instructor: Miss Fauzia** and **Miss Tehreem Course:** CS-116: Object Oriented Programming, Department of Computer & Information Systems Engineering. The primary source for understanding fundamental principles of OOP, class structure design, and complex engineering problem-solving strategies applied in this project.